
Series de Tiempo

Análisis, Modelado y Pronóstico

Vania Paola Juárez Mora

Notebook documentado – Python / Jupyter

Junio 2025

Contenido del documento

- Datos de vivienda en California
- Precios de cierre de Amazon (AMZN)
- Procesos AR(1) y Ruido Blanco
- Random Walk y Diferenciación
- Modelos MA y ARIMA
- Selección automática con `auto_arima`
- Modelo SARIMA / SARIMAX
- Distribución Binomial
- Modelos GARCH y Log-Retornos
- Pronóstico con Facebook Prophet

Contents

1	Introducción a las Series de Tiempo	2
2	Exploración de Datos – California Housing	2
2.1	Variables del dataset	2
3	Descarga de Precios de Amazon con yfinance	3
3.1	Visualización con Plotly	3
4	Proceso Autorregresivo AR(1)	3
4.1	Propiedades teóricas	4
4.2	Simulación en Python	4
5	Random Walk y Diferenciación	4
5.1	Propiedades	5
5.2	Diferenciación: de no estacionario a estacionario	5
6	Proceso de Media Móvil MA(q)	6
6.1	Función de autocorrelación del MA(1)	6
7	Modelos ARIMA	6
7.1	Generación de serie ARMA(1,1) sintética	7
7.2	Criterios de Información: AIC y BIC	7
7.2.1	Criterio de Akaike (AIC)	7
7.2.2	Criterio Bayesiano de Schwarz (BIC)	7
7.3	Ajuste de múltiples modelos ARIMA	7
7.3.1	Resultados de la comparación	8
7.4	Identificación gráfica: ACF y PACF	8
7.5	Diagnóstico de residuos	9
7.5.1	Q-Q Plot	9
7.5.2	Gráfica de residuos	9
8	Selección Automática: auto_arima	9
8.0.1	Resultado del auto_arima	10
8.1	Pronóstico con ambos modelos	10
9	Modelo SARIMA / SARIMAX	11
9.1	Modelo Airline: SARIMA(0, 1, 1)(0, 1, 1) ₁₂	11
9.2	Generación y ajuste de la serie SARIMAX	11
10	Distribución Binomial	12
11	Análisis de Volatilidad – Log-Retornos y GARCH	13
11.1	Descarga de datos recientes	13
11.2	Log-Retornos	13
11.3	ACF de log-retornos y de sus cuadrados	13

12 Pronóstico con Facebook Prophet	14
12.1 Ejemplo 1 – Serie sintética genérica	14
12.2 Ejemplo 2 – Precios de cierre de Amazon	15
12.3 Interpretación de los componentes	15
13 Resumen y Conclusiones	16

1. Introducción a las Series de Tiempo

Una **serie de tiempo** es una secuencia de observaciones registradas en instantes de tiempo sucesivos y (generalmente) equiespaciados. Formalmente:

Definición: Serie de Tiempo

Sea $\{X_t\}_{t \in \mathcal{T}}$ una colección de variables aleatorias indexada por el conjunto de tiempo $\mathcal{T} \subseteq \mathbb{Z}$. Decimos que $\{X_t\}$ es una **serie de tiempo** (o proceso estocástico en tiempo discreto).

El análisis de series de tiempo tiene como objetivo:

- **Describir** la estructura dinámica de los datos.
- **Modelar** la dependencia temporal entre observaciones.
- **Pronosticar** valores futuros.
- **Controlar** procesos que evolucionan en el tiempo.

2. Exploración de Datos – California Housing

La primera celda del notebook carga el conjunto de datos *California Housing Training*, un conjunto de referencia que contiene variables socioeconómicas y geográficas de bloques de viviendas en California.

```
1 import pandas as pd
2 ventas = pd.read_csv("california_housing_train.csv")
3 ventas.head(51)
```

Listing 1: Carga y vista previa del dataset

2.1. Variables del dataset

Variable	Tipo	Descripción
longitude	float	Longitud geográfica
latitude	float	Latitud geográfica
housing_median_age	float	Edad mediana de viviendas en el bloque
total_rooms	float	Total de habitaciones en el bloque
total_bedrooms	float	Total de dormitorios en el bloque
population	float	Población del bloque
households	float	Número de hogares
median_income	float	Ingreso mediano (en decenas de miles USD)
median_house_value	float	Valor mediano de la vivienda (USD)

3. Descarga de Precios de Amazon con yfinance

```
1 import yfinance as yf
2 amzn = yf.download("AMZN", end="2026-01-01")
3 amzn
```

Listing 2: Descarga de datos históricos de Amazon

```
1 precios_amz = amzn["Close"]
2 precios_amz.columns = ['Close']
3 precios_amz
```

Listing 3: Extracción de precios de cierre

La biblioteca `yfinance` permite descargar datos históricos del mercado financiero directamente desde Yahoo Finance. El objeto devuelto es un `DataFrame` de pandas con un índice de tipo `DatetimeIndex`.

3.1. Visualización con Plotly

```
1 import plotly.express as px
2
3 df_plot = precios_amz.reset_index()
4 fig = px.line(
5     df_plot,
6     x=df_plot.index.name,
7     y='Close',
8     title='Precios de Cierre de Amazon (AMZN)',
9     labels={'index': 'Fecha', 'Close': 'Precio de Cierre'},
10    color_discrete_sequence=['pink']
11 )
```

Listing 4: Gráfica interactiva de precios de cierre

4. Proceso Autorregresivo AR(1)

Definición: Proceso AR(1)

Un proceso autorregresivo de orden 1, denotado $AR(1)$, se define como:

$$X_t = \mu + \phi (X_{t-1} - \mu) + \varepsilon_t, \quad \varepsilon_t \sim \text{WN}(0, \sigma^2)$$

donde $|\phi| < 1$ es condición necesaria y suficiente para la **estacionariedad**.

4.1. Propiedades teóricas

$$\begin{aligned}\mathbb{E}[X_t] &= \mu \\ \text{Var}(X_t) &= \frac{\sigma^2}{1 - \phi^2} \\ \text{Cov}(X_t, X_{t-k}) &= \phi^{|k|} \frac{\sigma^2}{1 - \phi^2}\end{aligned}$$

4.2. Simulación en Python

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # Parametros del proceso AR(1)
6 phi = 0.7 # coeficiente autorregresivo (|phi| < 1 ->
    estacionario)
7 mu = 0 # media del proceso
8 sigma = 1 # desviacion estandar del ruido blanco
9 n_points = 200
10
11 ts = np.zeros(n_points)
12 white_noise = np.random.normal(loc=0, scale=sigma, size=n_points)
13
14 # Simulacion recursiva
15 for t in range(1, n_points):
16     ts[t] = phi * ts[t-1] + white_noise[t]
```

Listing 5: Simulación de un proceso AR(1)

Observación

Con $\phi = 0.7$ el proceso es **estacionario**: la serie oscila alrededor de cero con varianza constante $\sigma^2/(1 - 0.49) \approx 1.96$. Si $|\phi| \geq 1$ el proceso sería no estacionario (explosivo o random walk).

5. Random Walk y Diferenciación

Definición: Random Walk

Un **paseo aleatorio** (random walk) es un proceso $\{X_t\}$ que satisface:

$$X_t = X_{t-1} + Z_t, \quad Z_t \stackrel{\text{iid}}{\sim} N(0, \sigma^2), \quad X_0 = 0$$

5.1. Propiedades

$$X_t = \sum_{s=1}^t Z_s$$

$$\mathbb{E}[X_t] = 0$$

$$\text{Var}(X_t) = t\sigma^2 \quad \longrightarrow \quad \text{no estacionario}$$

```

1 import numpy as np
2 import plotly.express as px
3
4 np.random.seed(42)
5 T = 200
6 sigma = 1
7
8 Z = np.random.normal(0, sigma, T)
9
10 X = np.zeros(T)
11 for t in range(1, T):
12     X[t] = X[t-1] + Z[t]
13
14 fig = px.line(
15     x=np.arange(T), y=X,
16     title="Random Walk: X_t = X_{t-1} + Z_t",
17     labels={"x": "Tiempo (t)", "y": "X_t"}
18 )
19 fig.update_traces(line=dict(color="#ff1493", width=3))

```

Listing 6: Simulación de un Random Walk

5.2. Diferenciación: de no estacionario a estacionario

La **primera diferencia** $\Delta X_t = X_t - X_{t-1}$ convierte un random walk en ruido blanco:

$$\Delta X_t = Z_t \sim N(0, \sigma^2) \quad \Rightarrow \quad \text{proceso estacionario}$$

```

1 from statsmodels.tsa.stattools import adfuller, kpss
2
3 # Primera diferencia
4 dX = np.diff(X)
5
6 # Test de Dickey-Fuller aumentado
7 adf_result = adfuller(dX)
8 print(f"ADF stat: {adf_result[0]:.4f}, p-value:
9       {adf_result[1]:.4f}")

```

Listing 7: Diferenciación del Random Walk

6. Proceso de Media Móvil MA(q)

Definición: Proceso MA(1)

Un proceso de **media móvil de orden 1**, MA(1), se define como:

$$X_t = Z_t + \theta_1 Z_{t-1}, \quad Z_t \sim \text{WN}(0, \sigma^2)$$

6.1. Función de autocorrelación del MA(1)

$$\rho_k = \begin{cases} 1 & k = 0 \\ \frac{\theta_1}{1 + \theta_1^2} & k = 1 \\ 0 & k \geq 2 \end{cases}$$

La ACF se **corta** después del rezago q (propiedad distintiva del MA).

```

1 import numpy as np
2 import plotly.express as px
3
4 np.random.seed(42)
5 T = 200
6 Z = np.random.normal(0, 1, T)
7 Z_lag = np.roll(Z, 1); Z_lag[0] = 0
8
9 # Modelo 1: coeficiente theta = 2 (no invertible)
10 X1 = Z + 2 * Z_lag
11
12 # Modelo 2: coeficiente theta = 0.5 (invertible)
13 X2 = Z + 0.5 * Z_lag

```

Listing 8: Simulación de dos modelos MA(1)

Observación

El **Modelo 1** con $\theta_1 = 2$ *no es invertible* ($|\theta_1| > 1$). El **Modelo 2** con $\theta_1 = 0.5$ sí es invertible. La invertibilidad garantiza la unicidad de la representación MA.

7. Modelos ARIMA**Definición: ARIMA(p,d,q)**

Un modelo **ARIMA**(p, d, q) combina:

- p : orden autorregresivo (AR)
- d : orden de diferenciación (I)
- q : orden de media móvil (MA)

El modelo se especifica como:

$$\phi(B) (1 - B)^d X_t = \theta(B) \varepsilon_t$$

donde B es el operador de rezago ($BX_t = X_{t-1}$), y:

$$\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p, \quad \theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$$

7.1. Generación de serie ARMA(1,1) sintética

```

1 import numpy as np
2 import pandas as pd
3 from statsmodels.tsa.arima_process import arma_generate_sample
4
5 # Coeficientes AR: [1, -phi]
6 ar_params = np.array([1, -0.7])
7
8 # Coeficientes MA: [1, theta]
9 ma_params = np.array([1, 0.5])
10
11 n_samples = 200
12 arima_series = pd.Series(
13     arma_generate_sample(ar=ar_params, ma=ma_params,
14                          nsample=n_samples)
15 )

```

Listing 9: Generación de serie ARMA(1,1)

7.2. Criterios de Información: AIC y BIC

Para seleccionar el orden óptimo (p, d, q) se utilizan dos criterios:

7.2.1. Criterio de Akaike (AIC)

$$\text{AIC} = 2k - 2\ln(\hat{L})$$

donde k es el número de parámetros libres y $\hat{L}$ es el valor máximo de la función de verosimilitud.

7.2.2. Criterio Bayesiano de Schwarz (BIC)

$$\text{BIC} = k \ln(n) - 2\ln(\hat{L})$$

donde n es el tamaño de la muestra. El BIC penaliza más fuertemente la complejidad del modelo.

Regla de selección: se prefiere el modelo con el *menor* valor de AIC (o BIC).

7.3. Ajuste de múltiples modelos ARIMA

```

1 from statsmodels.tsa.arima.model import ARIMA
2
3 # ARIMA(1,0,0)
4 model_100 = ARIMA(arima_series, order=(1, 0, 0)).fit()

```

```

5
6 # ARIMA(0,0,1)
7 model_001 = ARIMA(arima_series, order=(0, 0, 1)).fit()
8
9 # ARIMA(1,0,1)
10 model_101 = ARIMA(arima_series, order=(1, 0, 1)).fit()
11
12 # Comparacion
13 print(f"ARIMA(1,0,0): AIC={model_100.aic:.3f},
      BIC={model_100.bic:.3f}")
14 print(f"ARIMA(0,0,1): AIC={model_001.aic:.3f},
      BIC={model_001.bic:.3f}")
15 print(f"ARIMA(1,0,1): AIC={model_101.aic:.3f},
      BIC={model_101.bic:.3f}")

```

Listing 10: Ajuste y comparación de modelos ARIMA

7.3.1. Resultados de la comparación

Modelo	AIC	BIC
ARIMA(1,0,0)	359.151	369.046
ARIMA(0,0,1)	353.439	363.334
ARIMA(1,0,1)	291.866	305.059

El modelo **ARIMA(1,0,1)** obtiene los menores valores de AIC y BIC, siendo el más adecuado para la serie sintética generada.

7.4. Identificación gráfica: ACF y PACF

Interpretación de ACF y PACF

- **ACF (Función de Autocorrelación)**: mide la correlación entre X_t y X_{t-k} .
 $\hookrightarrow$ Para un AR(p) decae exponencialmente; para un MA(q) se corta en q .
- **PACF (Función de Autocorrelación Parcial)**: mide la correlación entre X_t y X_{t-k} eliminando la influencia de los rezagos intermedios.
 $\hookrightarrow$ Para un AR(p) se corta en p ; para un MA(q) decae exponencialmente.

```

1 import matplotlib.pyplot as plt
2 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
3
4 fig_acf, ax_acf = plt.subplots(figsize=(12, 6))
5 plot_acf(arima_series, lags=40, color='#ff1493',
6         title='Funcion de Autocorrelacion (ACF)', ax=ax_acf)
7
8 fig_pacf, ax_pacf = plt.subplots(figsize=(12, 6))
9 plot_pacf(arima_series, lags=40, color='#ff1493',
10         title='Funcion de Autocorrelacion Parcial (PACF)',
11         ax=ax_pacf)
11 plt.show()

```

Listing 11: Gráficas ACF y PACF

7.5. Diagnóstico de residuos

7.5.1. Q-Q Plot

El **Q-Q plot** compara los cuantiles de los residuos contra los cuantiles de una distribución normal. Si los puntos se alinean sobre la recta de 45, los residuos son aproximadamente normales.

```
1 import statsmodels.api as sm
2
3 residuals = optimal_model_auto.resid()
4
5 fig_qq, ax_qq = plt.subplots(figsize=(10, 6))
6 sm.qqplot(residuals, line='45', ax=ax_qq,
7           markerfacecolor='#ff1493', markeredgecolor='#ff1493')
8 ax_qq.set_title('Q-Q Plot de Residuos del Modelo Optimo')
9 plt.show()
```

Listing 12: Q-Q plot de residuos

7.5.2. Gráfica de residuos

Los residuos de un buen modelo deben presentar:

1. **Aleatoriedad**: sin patrones visibles alrededor de cero.
2. **Media cero**: $\bar{\varepsilon} \approx 0$.
3. **Homocedasticidad**: varianza constante a lo largo del tiempo.
4. **Ausencia de autocorrelación**: confirmada por el test de Ljung-Box.

```
1 residuals = optimal_model_auto.resid()
2
3 plt.figure(figsize=(14, 7), facecolor='#ffe6f2')
4 plt.plot(residuals, color='#ff1493', linewidth=1.5)
5 plt.axhline(0, color='grey', linestyle='--', linewidth=0.8)
6 plt.title('Residuos del Modelo ARIMA Optimo')
7 plt.xlabel('Tiempo')
8 plt.ylabel('Residuo')
9 plt.show()
```

Listing 13: Gráfica de residuos

8. Selección Automática: auto_arima

La función `auto_arima` de la librería `pmdarima` automatiza la búsqueda del orden óptimo (p, d, q) evaluando múltiples combinaciones mediante AIC o BIC.

```
1 get_ipython().system('pip install pmdarima')
2 import pmdarima as pm
```

Listing 14: Instalación e importación de pmdarima

```

1 optimal_model_auto = pm.auto_arima(
2     arima_series,
3     trace=True,                # muestra el proceso de busqueda
4     suppress_warnings=False,
5     stepwise=True,            # busqueda por pasos (mas robusta)
6     seasonal=False,          # serie no estacional
7     d=0,                      # serie ya estacionaria
8     information_criterion='aic',
9     error_action='warn'
10 )
11 print(optimal_model_auto.summary())

```

Listing 15: Ejecución de auto_arima

8.0.1. Resultado del auto_arima

Modelo	AIC	BIC
ARIMA(2,0,1) (auto)	279.620	292.813
ARIMA(1,0,1) (manual)	291.866	305.059

El modelo **ARIMA(2,0,1)** encontrado automáticamente supera al seleccionado manualmente.

8.1. Pronóstico con ambos modelos

```

1 import plotly.graph_objects as go
2
3 # Pronostico manual ARIMA(1,0,1)
4 forecast_steps = 20
5 forecast_manual = model_101.predict(
6     start=len(arima_series),
7     end=len(arima_series) + forecast_steps - 1
8 )
9
10 # Pronostico auto_arima ARIMA(2,0,1)
11 forecast_auto =
12     optimal_model_auto.predict(n_periods=forecast_steps)
13
14 # Grafica comparativa
15 fig = go.Figure()
16 fig.add_trace(go.Scatter(x=arima_series.index,
17     y=arima_series.values,
18     name='Serie Original',
19     line=dict(color='blue'))))
20 fig.add_trace(go.Scatter(x=forecast_manual.index,
21     y=forecast_manual.values,
22     name='ARIMA(1,0,1)',
23     line=dict(color='red', dash='dot'))))
24 fig.add_trace(go.Scatter(x=forecast_auto.index,
25     y=forecast_auto.values,
26     name='ARIMA(2,0,1)',
27     line=dict(color='green',
28     dash='dash'))))
29 fig.show()

```

Listing 16: Generación de pronósticos

9. Modelo SARIMA / SARIMAX

Definición: SARIMA(p, d, q)(P, D, Q) $_s$

El modelo SARIMA extiende el ARIMA incorporando componentes estacionales:

$$\Phi_P(B^s) \phi_p(B) (1 - B^s)^D (1 - B)^d X_t = \Theta_Q(B^s) \theta_q(B) \varepsilon_t$$

donde s es el período estacional (e.g., $s = 12$ para datos mensuales).

9.1. Modelo Airline: SARIMA(0, 1, 1)(0, 1, 1) $_{12}$

Este es el modelo estacional clásico para datos de pasajeros de aerolíneas:

$$(1 - B)(1 - B^{12}) X_t = (1 + \theta_1 B)(1 + \Theta_1 B^{12}) \varepsilon_t$$

9.2. Generación y ajuste de la serie SARIMAX

```
1 import numpy as np
2 import pandas as pd
3 from statsmodels.tsa.statespace.sarimax import SARIMAX
4
5 ma_coeff = 0.7 # coeficiente MA no estacional
6 seasonal_ma_coeff = 0.5 # coeficiente MA estacional
7 n_samples = 300
8 season_length = 12
9
10 np.random.seed(42)
11 # Generacion de la serie sintetica SARIMA(0,1,1)(0,1,1)12
12 # (Ver implementacion completa en el notebook)
```

Listing 17: Simulación de serie con estacionalidad

```
1 model_sarimax_fit = SARIMAX(
2     sarimax_synthetic_series,
3     order=(0, 1, 1),
4     seasonal_order=(0, 1, 1, 12),
5     enforce_stationarity=False,
6     enforce_invertibility=False
7 ).fit(dispatch=False)
8
9 print(model_sarimax_fit.summary())
```

Listing 18: Ajuste del modelo SARIMAX

```
1 n_forecast_steps = 30
2 forecast_sarimax = model_sarimax_fit.predict(
3     start=len(sarimax_synthetic_series),
4     end=len(sarimax_synthetic_series) + n_forecast_steps - 1
```

5)

Listing 19: Pronóstico SARIMAX

10. Distribución Binomial

Definición: Variable Aleatoria Binomial

Sea $X \sim \text{Binomial}(n, p)$. La función de masa de probabilidad es:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n$$

$$\mathbb{E}[X] = np$$

$$\text{Var}(X) = np(1 - p)$$

```

1 import numpy as np
2 from scipy.stats import binom
3 import pandas as pd
4
5 # Parametros
6 n_scipy = 10
7 p_scipy = 0.5
8
9 # 30 muestras de la distribucion binomial
10 binomial_samples = binom.rvs(n=n_scipy, p=p_scipy, size=30)
11 print(binomial_samples)
12
13 # Funcion de masa de probabilidad
14 k_values = np.arange(0, n_scipy + 1)
15 pmf_values = binom.pmf(k_values, n_scipy, p_scipy)
16
17 pmf_df = pd.DataFrame({
18     'k': k_values,
19     'P(X=k)': pmf_values
20 })
21 print(pmf_df)

```

Listing 20: Muestreo y PMF Binomial

```

1 import matplotlib.pyplot as plt
2
3 plt.figure(figsize=(10, 6))
4 plt.hist(binomial_samples, bins=np.arange(n_scipy+2)-0.5,
5         edgecolor='black', color='#ff1493')
6 plt.title(f'Histograma Binomial (n={n_scipy}, p={p_scipy})',
7         fontsize=16, color='#cc0066')
8 plt.xlabel('Numero de Exitos'); plt.ylabel('Frecuencia')
9 plt.xticks(np.arange(n_scipy + 1))

```

```

10 plt.grid(axis='y', linestyle='--', alpha=0.7)
11 plt.show()

```

Listing 21: Histograma de la distribución binomial

11. Análisis de Volatilidad – Log-Retornos y GARCH

11.1. Descarga de datos recientes

```

1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4
5 amzn = yf.download("AMZN", start="2010-01-01")
6 amzn_precios = amzn["Close"]

```

Listing 22: Descarga de AMZN desde 2010

11.2. Log-Retornos

Los **log-retornos** son la transformación estándar en finanzas para analizar rendimientos:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

```

1 amzn_precios_df = amzn_precios.reset_index()
2 amzn_precios_df.columns = ['Date', 'Close']
3
4 amzn_precios_df['log_return'] = np.log(
5     amzn_precios_df['Close'] / amzn_precios_df['Close'].shift(1)
6 )
7 log_returns = amzn_precios_df['log_return'].dropna()

```

Listing 23: Cálculo de log-retornos

11.3. ACF de log-retornos y de sus cuadrados

La presencia de autocorrelación significativa en r_t^2 es indicador de **efectos ARCH/GARCH** (heterocedasticidad condicional).

```

1 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
2
3 fig, axes = plt.subplots(2, 1, figsize=(14, 10))
4 plot_acf(log_returns, lags=40, ax=axes[0],
5         title='ACF de Log Rendimientos de Amazon')
6 plot_pacf(log_returns, lags=40, ax=axes[1],
7         title='PACF de Log Rendimientos de Amazon')
8 plt.tight_layout(); plt.show()

```

Listing 24: ACF y PACF de log-retornos

12. Pronóstico con Facebook Prophet

Prophet es un modelo de series de tiempo desarrollado por Meta (Facebook) que descompone la serie en:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t$$

donde:

- $g(t)$: componente de **tendencia** (lineal o logística)
- $s(t)$: componente de **estacionalidad** (anual, semanal, diaria)
- $h(t)$: efecto de **festivos y eventos** especiales
- ε_t : término de error

12.1. Ejemplo 1 – Serie sintética genérica

```

1 import pandas as pd
2 import numpy as np
3
4 np.random.seed(42)
5 dates = pd.date_range(start='2023-01-01', periods=730, freq='D')
6
7 trend = np.linspace(0, 100, len(dates))
8 yearly_seasonality = 20 * np.sin(np.arange(len(dates)) *
9     (2*np.pi/365.25))
10 weekly_seasonality = 5 * np.sin(np.arange(len(dates)) *
11     (2*np.pi/7))
12 noise = np.random.normal(0, 5, len(dates))
13
14 y_generic = trend + yearly_seasonality + weekly_seasonality + noise
15
16 df_generic_prophet = pd.DataFrame({'ds': dates, 'y': y_generic})

```

Listing 25: Generación de serie sintética para Prophet

```

1 from prophet import Prophet
2
3 model_generic_prophet = Prophet(
4     yearly_seasonality=True,
5     weekly_seasonality=True
6 )
7 model_generic_prophet.fit(df_generic_prophet)
8
9 future_generic =
10     model_generic_prophet.make_future_dataframe(periods=30)
11 forecast_generic = model_generic_prophet.predict(future_generic)
12
13 # Visualización
14 model_generic_prophet.plot(forecast_generic)
15 model_generic_prophet.plot_components(forecast_generic)

```

Listing 26: Ajuste y pronóstico con Prophet (serie genérica)

12.2. Ejemplo 2 – Precios de cierre de Amazon

```

1 # Prophet requiere columnas 'ds' y 'y'
2 df_prophet_amzn = amzn_precios_df[['Date', 'Close']].rename(
3     columns={'Date': 'ds', 'Close': 'y'})
4 )
5 print(df_prophet_amzn.head())

```

Listing 27: Preprocesamiento para Prophet (Amazon)

```

1 model_prophet_amzn = Prophet(
2     yearly_seasonality=True,
3     weekly_seasonality=True,
4     daily_seasonality=False
5 )
6 model_prophet_amzn.fit(df_prophet_amzn)
7
8 future_amzn =
9     model_prophet_amzn.make_future_dataframe(periods=30)
10 forecast_amzn = model_prophet_amzn.predict(future_amzn)

```

Listing 28: Ajuste del modelo Prophet para Amazon

```

1 import matplotlib.pyplot as plt
2
3 # Pronostico general
4 fig_amzn = model_prophet_amzn.plot(forecast_amzn)
5 plt.title('Prophet Forecast -- Amazon Close Prices',
6     color='#cc0066', fontsize=16)
7
8 # Componentes (tendencia, estacionalidad anual, estacionalidad
9     semanal)
10 fig_comp = model_prophet_amzn.plot_components(forecast_amzn)
11 plt.show()

```

Listing 29: Visualización del pronóstico de Amazon

12.3. Interpretación de los componentes

Componente	Interpretación
Tendencia (Trend)	Dirección general del precio a largo plazo. Prophet detecta <i>change points</i> automáticamente.
Estacionalidad anual	Patrones que se repiten cada año (e.g., reportes trimestrales).
Estacionalidad semanal	Patrones dentro de la semana (e.g., mayor actividad lunes-viernes).

Observación

Prophet es robusto ante datos faltantes y valores atípicos, pero **no modela la volatilidad** (heterocedasticidad). Para mercados financieros, se recomienda complementarlo con modelos GARCH para el análisis de riesgo.

13. Resumen y Conclusiones

Este notebook cubre el flujo completo de análisis de series de tiempo:

1. **Exploración de datos:** carga y visualización de datasets estructurados (California Housing, AMZN).
2. **Modelos base:** simulación de procesos AR(1), MA(1) y Random Walk, con estudio de sus propiedades de estacionariedad.
3. **Familia ARIMA:** generación, ajuste, selección (AIC/BIC) y diagnóstico de residuos.
4. **auto_arima:** búsqueda automática del orden óptimo, mejorando el modelo de ARIMA(1,0,1) a ARIMA(2,0,1).
5. **SARIMA/SARIMAX:** extensión al caso estacional con $s = 12$.
6. **Análisis financiero:** log-retornos y ACF para detección de efectos GARCH.
7. **Prophet:** pronóstico descompuesto en tendencia y estacionalidad para datos de Amazon.

Herramientas utilizadas

pandas, numpy	Manipulación de datos
statsmodels	Modelos ARIMA, SARIMA, ACF/PACF
pmdarima	Selección automática (<code>auto_arima</code>)
prophet	Pronóstico descompuesto
yfinance	Datos financieros en tiempo real
plotly, matplotlib, altair	Visualizaciones
scipy.stats	Distribuciones estadísticas